

AWS

How the core cloud services fit together — the illustrated field guide



with Cargo, your guide

Part of the Interview-Preparation illustrated series

Table of Contents

1. Regions and Availability Zones
2. IAM Roles: No Embedded Credentials
3. Auto Scaling: Add Capacity on Demand
4. S3 Storage Classes
5. VPC: Public and Private Subnets
6. RDS: Multi-AZ vs. Read Replicas
7. Lambda: No Server, No Traffic, No Cost
8. The Shared Responsibility Model
9. Cheat Sheet

This guide focuses on the handful of AWS concepts that show up in nearly every architecture decision. For the full breadth of services, see the companion 150-question AWS Q&A file.

CHAPTER 1

Regions and Availability Zones

AWS's infrastructure is organized into Regions -- large geographic areas like us-east-1 -- each containing multiple Availability Zones (AZs): physically separate data centers with independent power, cooling, and networking.

A single AZ can fail -- a power outage, a network issue, a natural disaster affecting that specific facility. If everything you run lives in just one AZ, that failure takes your whole application down with it. Spreading instances and data across multiple AZs within a Region means a single facility's outage only takes out a portion of your capacity, not all of it.

Regions and Availability Zones



Deploy across multiple AZs: one data center failing doesn't take everything down.

IN SHORT: Never deploy production workloads to a single AZ. Multi-AZ is the baseline, not an optional upgrade.

CHAPTER 2

IAM Roles: No Embedded Credentials

The classic mistake: hardcoding an AWS access key and secret directly into application code or a config file so an EC2 instance can talk to S3. Those credentials are long-lived, and if they ever leak -- a committed file, a misconfigured backup -- an attacker has standing access until someone notices and rotates them.

An IAM Role solves this differently. You attach a role to the EC2 instance itself; AWS delivers temporary, automatically-rotated credentials to it behind the scenes. There's nothing long-lived stored on disk to leak in the first place.

IAM Roles: No Embedded Credentials



The instance assumes a role and receives short-lived, auto-rotated credentials -- nothing long-lived to leak.

IN SHORT: Never embed long-term AWS credentials in code or on a server. Attach a role and let AWS hand out short-lived credentials instead.

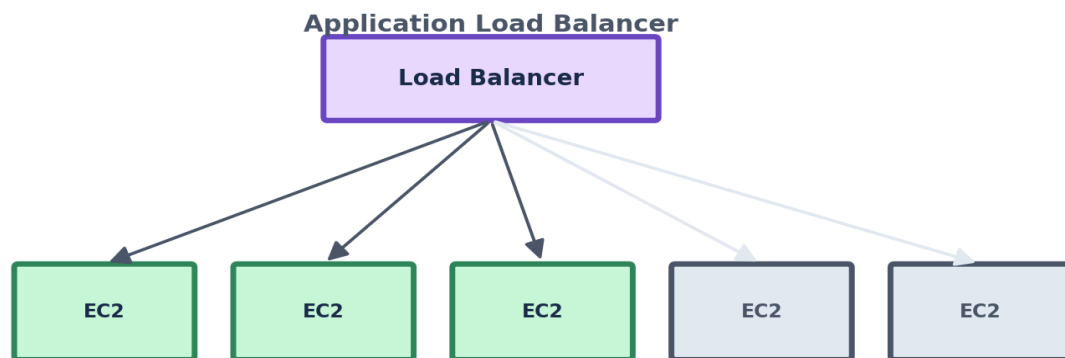
CHAPTER 3

Auto Scaling: Add Capacity on Demand

An Auto Scaling Group maintains a target number of EC2 instances, automatically replacing any that fail a health check, and adjusting that target count based on real demand -- more instances when a scaling policy detects rising load (like CPU utilization crossing a threshold), fewer once it drops back down.

A Load Balancer sits in front of the group, distributing incoming traffic across whichever instances are currently healthy -- neither the load balancer nor the application needs to know or care exactly how many instances currently exist.

Auto Scaling: Add Capacity on Demand



The Auto Scaling Group launches more EC2 instances as load increases, and terminates them again once demand drops.

IN SHORT: You set the scaling policy and the bounds. The Auto Scaling Group handles the actual launching, health-checking, and terminating continuously.

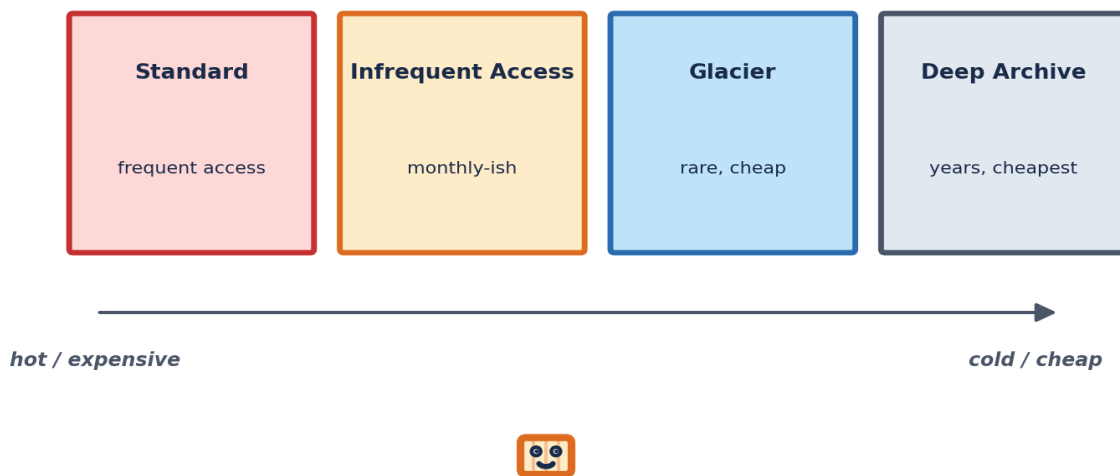
CHAPTER 4

S3 Storage Classes

Not all data is accessed equally often, and S3 lets you pay accordingly. Standard storage is optimized for data you access frequently. As access frequency drops, moving data to a colder tier -- Infrequent Access, then Glacier, then Deep Archive -- significantly reduces storage cost, at the price of a retrieval fee and, for the coldest tiers, a genuine wait (hours) before archived data becomes available again.

Lifecycle policies can automate this entirely -- e.g., "move objects to Infrequent Access after 30 days, then to Glacier after 90."

S3 Storage Classes: Pick Your Temperature



IN SHORT: Match the storage class to actual access pattern. Paying Standard-tier prices for data nobody has touched in a year is pure waste.

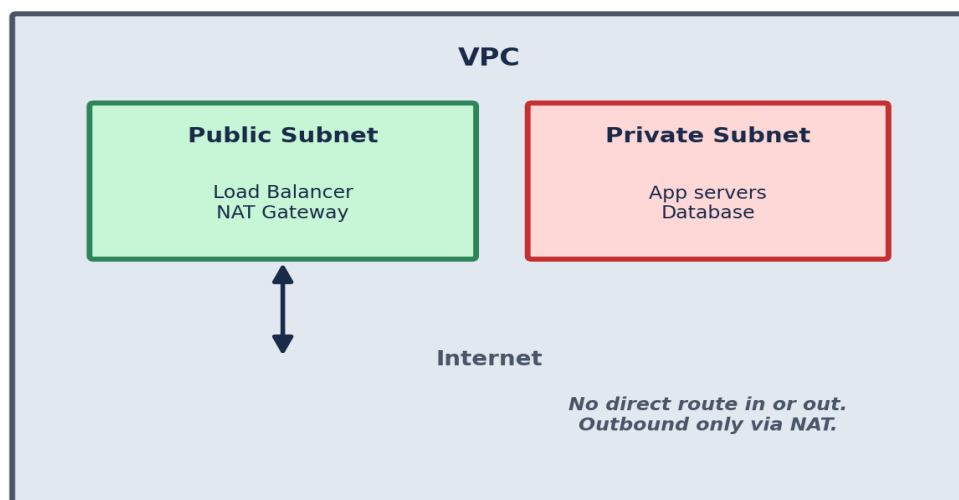
CHAPTER 5

VPC: Public and Private Subnets

A VPC is your own private, isolated network within AWS. Within it, subnets are typically split into two categories: public subnets, which have a route to an Internet Gateway (so resources there can be reached from, and reach, the internet directly), and private subnets, which have no such direct route.

The standard pattern: load balancers and NAT gateways live in public subnets; application servers and databases -- the things that should never be directly reachable from the internet -- live in private subnets, reaching the internet for outbound-only needs (like downloading updates) through the NAT gateway.

VPC: Public and Private Subnets



IN SHORT: Public subnet = internet-facing. Private subnet = everything that should never be directly reachable from outside your VPC.

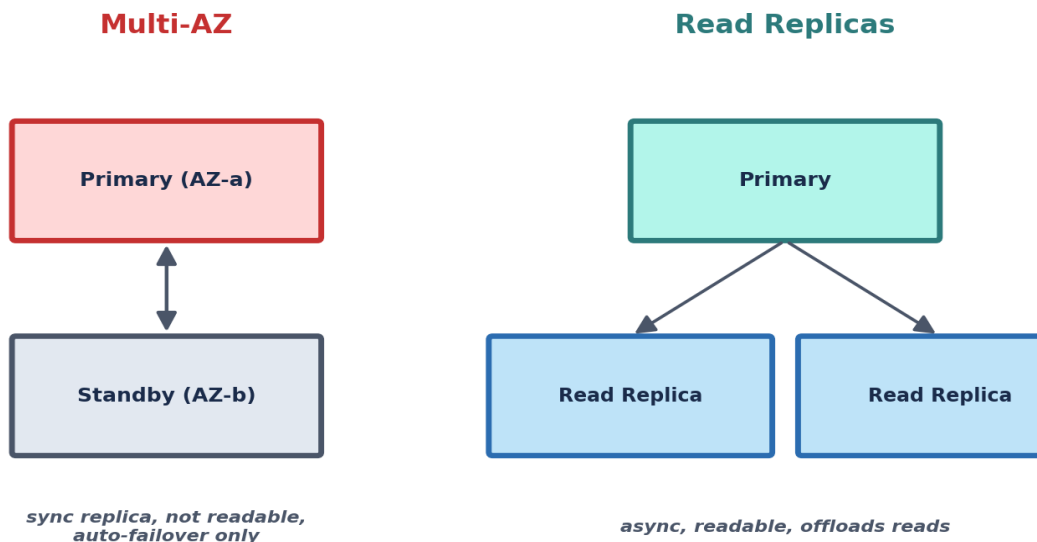
CHAPTER 6

RDS: Multi-AZ vs. Read Replicas

These solve two genuinely different problems, and confusing them is a common mistake. A Multi-AZ deployment keeps a synchronously-replicated standby database in a different AZ purely for failover -- it's not readable, and its only job is to take over automatically if the primary fails.

Read Replicas are asynchronously-replicated, independently readable copies -- built to offload read traffic from an overloaded primary, not for automatic failover. You can, and often should, use both together: Multi-AZ for availability, read replicas for read-scaling.

Multi-AZ (Failover) vs. Read Replicas



IN SHORT: Multi-AZ = availability insurance, not readable. Read Replicas = read scaling, not automatic failover. Different tools for different problems.

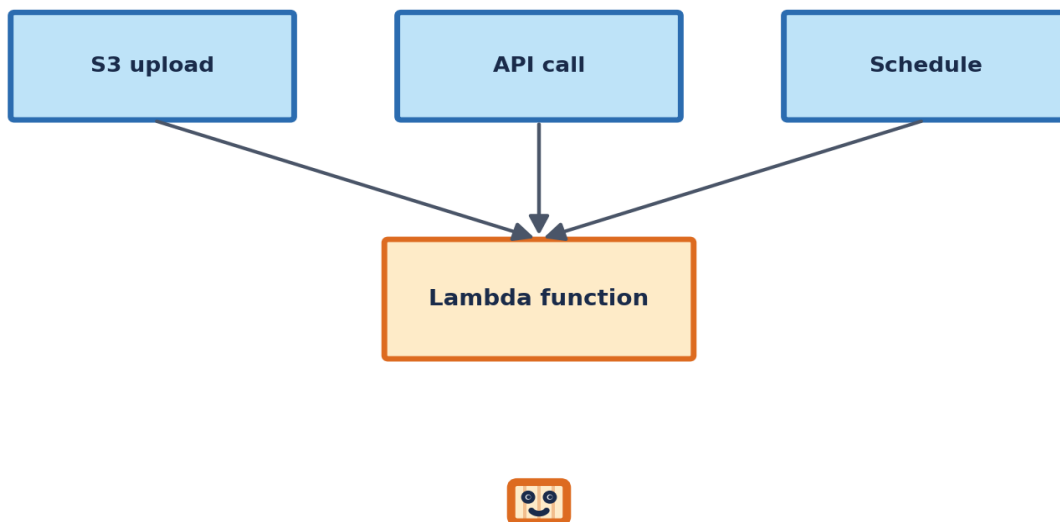
CHAPTER 7

Lambda: No Server, No Traffic, No Cost

Lambda runs your code in response to an event -- an S3 upload, an API call, a scheduled trigger -- without you provisioning or managing any server at all.

When there's no traffic, there's genuinely nothing running and nothing billed. When a burst of events arrives, AWS automatically spins up as many concurrent executions as needed. This is what "serverless" actually means in practice: not that there are no servers, but that you never think about them.

Lambda: No Server, No Traffic, No Cost



Runs only when triggered. Scales automatically. Pay per invocation, not per idle hour.

IN SHORT: Perfect for short-lived, event-driven, spiky workloads. Not a fit for long-running processes -- Lambda has a hard 15-minute execution limit.

CHAPTER 8

The Shared Responsibility Model

Security on AWS is a partnership, not something AWS handles entirely for you. AWS is responsible for security *of* the cloud -- the physical data centers, hardware, and the virtualization layer underneath everything you run.

You're responsible for security *in* the cloud -- how you configure IAM, whether you encrypt your data, how you set up security groups, and (for EC2 specifically) patching the operating system. A perfectly secure AWS data center doesn't help you if your S3 bucket is misconfigured to be publicly readable.

The Shared Responsibility Model



IN SHORT: AWS secures the infrastructure. You secure your configuration of it. Most real-world cloud security incidents happen on the customer's side of this line.

CHAPTER 9

Cheat Sheet

The core services, grouped by what they're for.

AWS CHEAT SHEET



Compute

EC2 - virtual servers

Lambda - serverless functions

ECS/EKS - containers

Auto Scaling Group - elastic capacity

Storage

S3 - object storage

EBS - block storage (per instance)

EFS - shared file storage

Database

RDS - managed relational DB

DynamoDB - managed NoSQL

ElastiCache - managed Redis/Memcached

Networking

VPC - isolated virtual network

Route 53 - DNS

CloudFront - CDN

ELB - load balancing

Security

IAM - identities & permissions

KMS - encryption keys

Secrets Manager - credentials